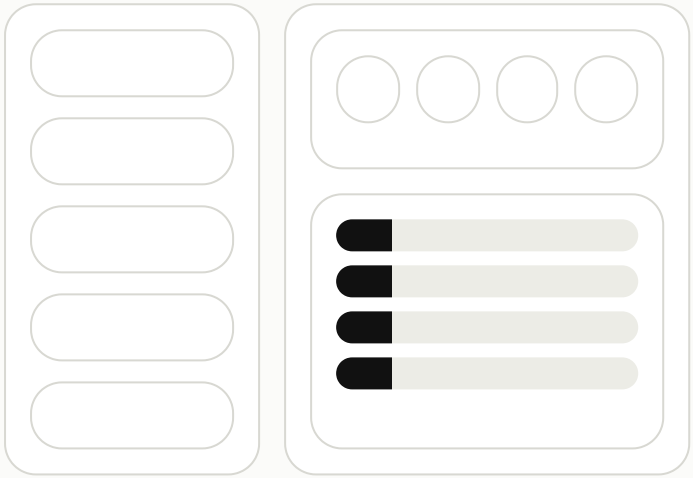
LODGE MANAGEMENT SOFTWARE

Product Documentation

Professional black and white lodge operations software built for a single-property launch, with room management, guest workflows, employee tracking, services, housekeeping, inventory, and Supabase-ready cloud persistence.

INTERFACE PREVIEW



PREPARED FOR

Owner / Reseller / Customer Demo

This document explains what is built, how it works, what technology is used, and how it can be hosted.

CURRENT SCOPE

- Single lodge operations software
- White theme default with black mode toggle
- Static hosting friendly architecture
- Supabase-ready cloud data flow

THEME MODES

2

White default and black optional mode

MAIN MODULES

13

Dashboard to user access control

ADD FORMS

7+

Guests, rooms, staff, stock and services

CLOUD READY

Yes

Supabase sync with local fallback support

1. Executive Summary

The application is a frontend-first Lodge Management Software built in React and Vite. It is designed to help one lodge manage daily operations such as room status, guest intake, employee records, service menus, expense tracking, inventory, housekeeping, billing, and admin access.

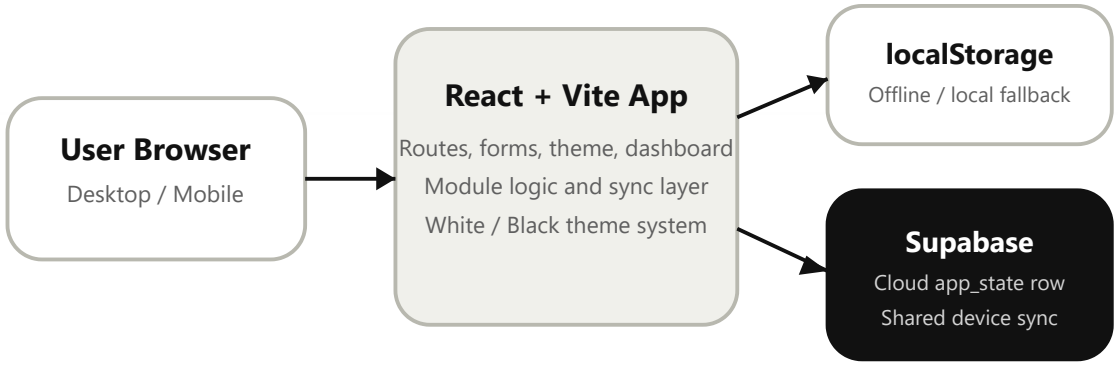
Business Goal

Launch quickly for a single property, demo easily to customers, and keep the software affordable to host and support.

Current Delivery Model

Static frontend hosting plus optional Supabase cloud storage, with local browser storage fallback when cloud sync is not configured.

SYSTEM ARCHITECTURE



2. Technology Stack

LAYER	TECHNOLOGY	PURPOSE	STATUS
Frontend	React 18	Build the user interface and module pages	Implemented
Build Tool	Vite 8	Fast development server and production build	Configured and verified
Language	TypeScript	Type safety and maintainability	Implemented
Routing	React Router DOM	Navigation between app modules	Implemented
Local Persistence	localStorage	Offline / local-only data storage	Implemented
Cloud Persistence	Supabase REST + Postgres	Shared multi-device lodge data	Prepared and environment based
Styling	Custom CSS	Professional black and white UI system	Implemented

Why React

Clean module separation and easy extension of business features over time.

Why Vite

Fast builds, easy static deployment, and low-friction developer workflow.

Why Supabase

Simple cloud sync for a small lodge launch without building a full custom backend first.

3. Functional Modules And Added Features

MODULE	MAIN PURPOSE	WHAT IT DOES	ADDED / UPDATED
Dashboard	Daily operations overview	Occupancy, revenue, expenses and alerts	Professional monochrome presentation
Guests	Guest lifecycle	Add guests, reserve room, check-in and check-out	Operational flow retained
Rooms	Room inventory management	Track room type, status, price and history	New add room form added
Services	Food and beverage planning	Breakfast and tea schedule with charges	New add service form added
Staff	Employee records	Attendance, shift and assigned task management	New add employee form added
Expenses	Cost control	Add and review expense records	Existing form retained
Housekeeping	Room cleaning workflow	Laundry, cleaning status and notes	New add housekeeping form added
Inventory	Supply management	Track stock, minimum levels and low stock alerts	New add inventory form added
Users	Admin controls	Create parent admins with limited permissions	Existing form retained

Theme Changes

- White theme set as the default
- Black theme available by toggle
- Blue-tinted glass styling removed
- Readable monochrome tables, cards and forms introduced

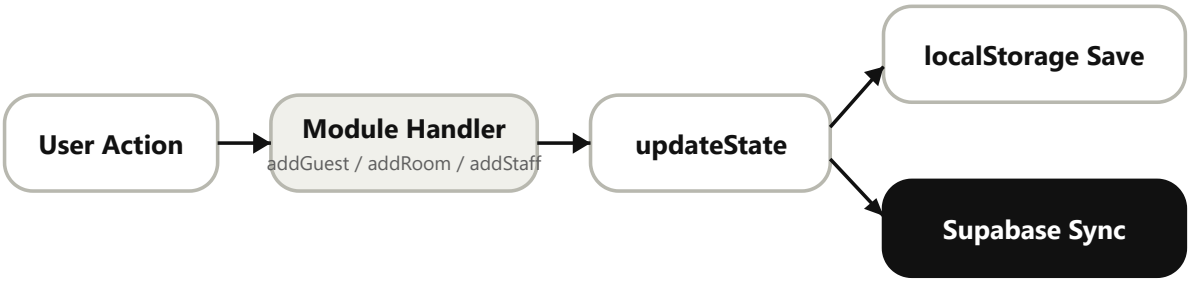
Cloud Readiness Changes

- Supabase environment template added
- Supabase SQL schema file added
- Remote sync helper introduced
- Sidebar sync status badge added

4. Function-Wise Logic Breakdown

FUNCTION / AREA	RESPONSIBILITY	PRACTICAL EFFECT
loadState()	Loads application data from local storage safely	Prevents crashes if stored JSON is damaged
saveState()	Saves latest application state locally	Keeps app usable on one device without cloud setup
updateState()	Central state writer for UI + local + remote sync	All feature changes move through one reliable path
toggleTheme()	Switches between white and black themes	Persistent visual preference for user
handleLogin()	Validates configured user credentials	Protects module access using role and permissions
addGuest()	Creates guest and suggests a matching room	Front-desk guest intake becomes faster
checkInGuest() / **checkOutGuest()**	Updates guest and room records together	Maintains room occupancy consistency
addRoom()	Adds room inventory records	Operator can expand room stock directly in the app
addService()	Adds meal service entries	Daily food plan and add-on billing can be maintained
addStaff()	Adds employee details and assignments	Staff roster can be extended without code changes
addHousekeeping()	Adds cleaning records per room	Room readiness can be tracked more clearly
addInventory()	Adds stock item records	Supply usage can be monitored as the lodge grows
remoteState.ts	Fetches and saves app state to Supabase	Allows data sharing across multiple devices

STATE UPDATE FLOW


User Action
Module Handler
addGuest / addRoom / addStaff
updateState
localStorage Save
Supabase Sync

5. Hosting, Database And Deployment

AREA	RECOMMENDED OPTION	REASON	OPERATIONAL NOTE
Frontend Hosting	Cloudflare Pages or Netlify	Good fit for static Vite builds	Easy GitHub-based deployment
Database	Supabase	Fastest cloud setup for a one-lodge app	Current code is already prepared for it
Current Data Model	`app_state` JSON row	Simple and low-friction for MVP use	Best for one lodge and early trials
Future Upgrade	Separate normalized tables	Better reporting, security, and growth	Recommended before scaling to many customers

Deployment Checklist

- Push code to GitHub
- Create Supabase project
- Run `supabase-schema.sql`
- Create environment variables
- Connect repo to hosting platform
- Build with `npm run build`
- Publish the `dist` folder

Environment Variables

VARIABLE	USE
`VITE_SUPABASE_URL`	Supabase project URL
`VITE_SUPABASE_ANON_KEY`	Supabase browser anon key
`VITE_APP_STATE_ROW_ID`	Single lodge row identifier

Current Code Quality Status

- TypeScript build passes
- Vite production build passes
- Remote sync edge case improved for first-time cloud setup
- Safer local storage parsing added to avoid startup failure

6. Recommendations Before Selling

PRIORITY	RECOMMENDATION	REASON
High	Secure Supabase before serious production rollout	The current sync model is deliberately simple for demo and MVP use
High	Use separate hosting / database project per customer	Improves isolation, maintainability and support handling
Medium	Add backup export tools	Useful for recovery and customer confidence
Medium	Normalize the database later	Improves reporting and long-term data flexibility

What Is Ready Now

Demo-ready UI, multi-module workflows, GitHub-ready project, and a print-ready product document.

What Is Best Next

Connect Supabase credentials, deploy to hosting, and tighten database rules before active customer usage.

Commercial Position

Suitable for a first lodge pilot, especially when bundled with yearly support and maintenance.